

Tasks

Apache Airflow 3.3.2

Origem: <https://airflow.apache.org/docs/apache-airflow/3.3.2/core-concepts/tasks.html>

Copia de estudo exportada em 21/09/2026. Texto original em ingles; formatacao e paginacao locais. Imagens e navegacao omitidas; consulte a origem para diagramas. Apache License 2.0; creditos em sources/.

Tasks

A Task is the basic unit of execution in Airflow. Tasks are arranged into Dags , and then have upstream and downstream dependencies set between them in order to express the order they should run in.

There are three basic kinds of Task:

Operators , predefined task templates that you can string together quickly to build most parts of your Dags.

Sensors , a special subclass of Operators which are entirely about waiting for an external event to happen.

A TaskFlow -decorated @task , which is a custom Python function packaged up as a Task.

Internally, these are all actually subclasses of Airflow's BaseOperator , and the concepts of Task and Operator are somewhat interchangeable, but it's useful to think of them as separate concepts - essentially, Operators and Sensors are templates , and when you call one in a Dag file, you're making a Task.

Relationships

The key part of using Tasks is defining how they relate to each other - their dependencies , or as we say in Airflow, their upstream and downstream tasks. You declare your Tasks first, and then you declare their dependencies second.

Note

We call the upstream task the one that is directly preceding the other task. We used to call it a parent task before. Be aware that this concept does not describe the tasks that are higher in the tasks hierarchy (i.e. they are not a direct parents of the task). Same definition applies to downstream task, which needs to be a direct child of the other task.

There are two ways of declaring dependencies - using the >> and << (bitshift) operators:

```
first_task >> second_task >> [third_task, fourth_task]
```

Or the more explicit set_upstream and set_downstream methods:

```
first_task.set_downstream(second_task)
third_task.set_upstream(second_task)
```

These both do exactly the same thing, but in general we recommend you use the bitshift operators, as they are easier to read in most cases.

By default, a Task will run when all of its upstream (parent) tasks have succeeded, but there are many ways of modifying this behaviour to add branching, to only wait for some upstream tasks, or to change behaviour based on where the current run is in history. For more, see Control Flow .

Tasks don't pass information to each other by default, and run entirely independently. If you want to pass information from one Task to another, you should use XComs .

Task Instances

Much in the same way that a Dag is instantiated into a Dag Run each time it runs, the tasks under a Dag are instantiated into Task Instances .

An instance of a Task is a specific run of that task for a given Dag (and thus for a given data interval). They are also the representation of a Task that has state , representing what stage of the lifecycle it is in.

The possible states for a Task Instance are:

none : The Task has not yet been queued for execution (its dependencies are not yet met)

scheduled : The scheduler has determined the Task's dependencies are met and it should run

queued : The task has been assigned to an Executor and is awaiting a worker

running : The task is running on a worker (or on a local/synchronous executor)

success : The task finished running without errors

restarting : The task was externally requested to restart when it was running

failed : The task had an error during execution and failed to run

skipped : The task was skipped due to branching, LatestOnly, or similar.

upstream_failed : An upstream task failed and the Trigger Rule says we needed it

up_for_retry : The task failed, but has retry attempts left and will be rescheduled.

up_for_reschedule : The task is a Sensor that is in reschedule mode

deferred : The task has been deferred to a trigger

awaiting_input : The task is a Human-in-the-loop task waiting for a human response. It is managed by the scheduler and uses neither a worker slot nor the triggerer.

removed : The task has vanished from the Dag since the run started

Ideally, a task should flow from none , to scheduled , to queued , to running , and finally to success .

When any custom Task (Operator) is running, it will get a copy of the task instance passed to it; as well as being able to inspect task metadata, it also contains methods for things like XComs .

Relationship Terminology

For any given Task Instance, there are two types of relationships it has with other instances.

Firstly, it can have upstream and downstream tasks:

```
task1 >> task2 >> task3
```

When a Dag runs, it will create instances for each of these tasks that are upstream/downstream of each other, but which all have the same data interval.

There may also be instances of the same task , but for different data intervals - from other runs of the same Dag. We call these previous and next - it is a different relationship to upstream and downstream !

Note

Some older Airflow documentation may still use “previous” to mean “upstream”. If you find an occurrence of this, please help us fix it!

Timeouts

If you want a task to have a maximum runtime, set its `execution_timeout` attribute to a `datetime.timedelta` value that is the maximum permissible runtime. This applies to all Airflow tasks, including sensors. `execution_timeout` controls the maximum time allowed for every execution. If `execution_timeout` is breached, the task times out and

AirflowTaskTimeout is raised.

In addition, sensors have a timeout parameter. This only matters for sensors in reschedule mode. timeout controls the maximum time allowed for the sensor to succeed. If timeout is breached, AirflowSensorTimeout will be raised and the sensor fails immediately without retrying.

The following SFTPSensor example illustrates this. The sensor is in reschedule mode, meaning it is periodically executed and rescheduled until it succeeds.

Each time the sensor pokes the SFTP server, it is allowed to take maximum 60 seconds as defined by execution_timeout .

If it takes the sensor more than 60 seconds to poke the SFTP server, AirflowTaskTimeout will be raised. The sensor is allowed to retry when this happens. It can retry up to 2 times as defined by retries .

From the start of the first execution, till it eventually succeeds (i.e. after the file 'root/test' appears), the sensor is allowed maximum 3600 seconds as defined by timeout . In other words, if the file does not appear on the SFTP server within 3600 seconds, the sensor will raise AirflowSensorTimeout . It will not retry when this error is raised.

If the sensor fails due to other reasons such as network outages during the 3600 seconds interval, it can retry up to 2 times as defined by retries . Retrying does not reset the timeout . It will still have up to 3600 seconds in total for it to succeed.

```
sensor = SFTPSensor(
    task_id="sensor",
    path="/root/test",
    execution_timeout=timedelta(seconds=60),
    timeout=3600,
    retries=2,
    mode="reschedule",
)
```

SLAs

The SLA feature from Airflow 2 has been removed in 3.0 and was replaced in Airflow 3.1 with Deadlines Alerts .

Special Exceptions

If you want to control your task's state from within custom Task/Operator code, Airflow provides two special exceptions you can raise:

AirflowSkipException will mark the current task as skipped

AirflowFailException will mark the current task as failed ignoring any remaining retry attempts

These can be useful if your code has extra knowledge about its environment and wants to fail/skip faster - e.g., skipping when it knows there's no data available, or fast-failing when it detects its API key is invalid (as that will not be fixed by a retry).

Retry Policies

By default, Airflow retries failed tasks with a fixed count and delay regardless of the error type. A retry policy lets you configure per-exception retry behaviour as a parameter on any task or operator, without modifying task code.

Define a policy with rules that map exception types to actions, then apply it to a task:

[airflow/example_dags/example_retry_policy.py](#) [source]

```
from airflow.sdk import DAG, ExceptionRetryPolicy, RetryAction, RetryRule, task

API_RETRY_POLICY = ExceptionRetryPolicy(
    rules=[
        RetryRule(
```

```

        exception="requests.exceptions.HTTPError",
        action=RetryAction.RETRY,
        retry_delay=timedelta(minutes=5),
        reason="Rate limit, backing off",
    ),
    RetryRule(
        exception="google.auth.exceptions.RefreshError",
        action=RetryAction.FAIL,
        reason="Auth failure, not retryable",
    ),
    RetryRule(
        exception=ConnectionError,
        action=RetryAction.RETRY,
        retry_delay=timedelta(seconds=30),
    ),
],
)

```

[airflow/example_dags/example_retry_policy.py \[source\]](#)

```

with DAG(
    dag_id="example_retry_policy",
    schedule=None,
    catchup=False,
    tags=["example", "retry_policy"],
):

    @task(retries=5, retry_delay=timedelta(minutes=1), retry_policy=API_RETRY_POLICY)
    def call_external_api():
        import requests

        response = requests.get("https://api.example.com/data")
        response.raise_for_status()
        return response.json()

    call_external_api()

```

How it works

The policy runs in the task worker process (never the scheduler) between catching the exception and deciding the task's next state. Each policy decision is logged in the task logs as `Retry policy decision action=<action> reason=<reason>`.

When a task fails, the policy evaluates the exception and returns one of three actions:

RETRY – retry the task, optionally with a custom delay that overrides `retry_delay`. The retry is still subject to the task's retries count – a policy can fail earlier but cannot extend past the configured maximum.

FAIL – fail immediately, skipping any remaining retries.

DEFAULT – fall through to the standard retry logic (retries count and `retry_delay`).

Rules are evaluated in order; the first matching rule wins. If no rule matches, the policy returns **DEFAULT** (standard retry behaviour).

Exception matching

Exception types can be specified as Python classes or dotted import path strings (e.g. `"requests.exceptions.HTTPError"`). String paths are validated at DAG parse time – a path without a dot raises `ValueError` immediately, and unresolvable paths produce a warning.

By default, rules use `isinstance` matching, so a rule for `OSError` also matches `ConnectionError` (a subclass). Set `match_subclasses=False` for exact type matching:

```
RetryRule(exception=OSError, match_subclasses=False) # only OSError, not subclasses
```

Composition with existing parameters

Parameter Behaviour when `retry_policy` is set

retries Still the maximum retry count. Policy can fail earlier but not exceed it.

retry_delay / retry_exponential_backoff / max_retry_delay Used when the policy returns DEFAULT or when RetryDecision.retry_delay is None.

on_retry_callback Fires on all retries, including policy-driven retries.

AirflowFailException Always takes precedence. The policy is never consulted for this exception.

Reusable policies

Define a policy once and share it across DAGs via default_args or a shared module:

airflow/example_dags/example_retry_policy.py [source]

```
# policies.py -- import in any DAG
STANDARD_RETRY_POLICY = ExceptionRetryPolicy(
    rules=[
        RetryRule(exception="requests.exceptions.HTTPError", action=RetryAction.FAIL),
        RetryRule(exception=ConnectionError, retry_delay=timedelta(seconds=10)),
    ],
)
```

Mapped tasks

Policies work with dynamic task mapping via .partial() . The policy applies per mapped task instance – if instance 2 of 10 hits FAIL, the other 9 continue independently:

```
@task.partial(retry_policy=my_policy).expand(input=[1, 2, 3])
def my_mapped_task(input): ...
```

The policy is set at the task level via .partial() ; all mapped instances share one policy. Per-index variation is not supported on .expand() , but the policy's evaluate() method receives the exception, try_number , and full context, so per-index branching can be done inside the policy if needed.

Custom retry policies

For advanced cases, subclass RetryPolicy and implement evaluate() . Subclassing is the right choice when you need to inspect exception attributes (status codes, response headers, payloads) that the declarative ExceptionRetryPolicy rules cannot capture. The method receives the exception, try number, max tries, and the full Airflow context (dag_run , params , etc.):

```
from datetime import timedelta

import requests

from airflow.sdk import RetryDecision, RetryPolicy

class HTTPStatusRetryPolicy(RetryPolicy):
    """Route retry decisions by HTTP status code, honouring Retry-After on 429s."""

    def evaluate(self, exception, try_number, max_tries, context=None):
        if isinstance(exception, requests.HTTPError) and exception.response is not None:
            status = exception.response.status_code
            if status == 429: # rate limited -- honour Retry-After header
                retry_after = int(exception.response.headers.get("Retry-After", 60))
                return RetryDecision.retry(retry_delay=timedelta(seconds=retry_after))
            if 500 <= status < 600: # server error -- worth retrying
                return RetryDecision.retry()
            if 400 <= status < 500: # client error -- not retryable
                return RetryDecision.fail(reason=f"HTTP {status}")
        return RetryDecision.default()
```

A second pattern uses the run context to make decisions based on how the DAG was triggered:

```
from airflow.sdk import RetryDecision, RetryPolicy

class BackfillAwareRetryPolicy(RetryPolicy):
```

```
"""Fail fast during backfills so historical errors surface immediately."""
```

```
def evaluate(self, exception, try_number, max_tries, context=None):  
    if context and context["dag_run"].run_type == "backfill":  
        return RetryDecision.fail(reason="Backfill run -- not retrying")  
    return RetryDecision.default()
```

Task Instance Heartbeat Timeout

No system runs perfectly, and task instances are expected to die once in a while.

TaskInstances may get stuck in a running state despite their associated jobs being inactive (for example if the TaskInstance's worker ran out of memory). Such tasks were formerly known as zombie tasks. Airflow will find these periodically, clean them up, and mark the TaskInstance as failed or retry it if it has available retries. The TaskInstance's heartbeat can timeout for many reasons, including:

The Airflow worker ran out of memory and was OOMKilled.

The Airflow worker failed its liveness probe, so the system (for example, Kubernetes) restarted the worker.

The system (for example, Kubernetes) scaled down and moved an Airflow worker from one node to another.

Reproducing task instance heartbeat timeouts locally

If you'd like to reproduce task instance heartbeat timeouts for development/testing processes, follow the steps below:

Set the below environment variables for your local Airflow setup (alternatively you could tweak the corresponding config values in airflow.cfg)

```
export AIRFLOW__SCHEDULER__TASK_INSTANCE_HEARTBEAT_SEC=600  
export AIRFLOW__SCHEDULER__TASK_INSTANCE_HEARTBEAT_TIMEOUT=2  
export AIRFLOW__SCHEDULER__TASK_INSTANCE_HEARTBEAT_TIMEOUT_DETECTION_INTERVAL=5
```

Have a Dag with a task that takes about 10 minutes to complete(i.e. a long-running task). For example, you could use the below Dag:

```
from airflow.sdk import dag  
from airflow.providers.standard.operators.bash import BashOperator  
from datetime import datetime  
  
@dag(start_date=datetime(2021, 1, 1), schedule="@once", catchup=False)  
def sleep_dag():  
    t1 = BashOperator(  
        task_id="sleep_10_minutes",  
        bash_command="sleep 600",  
    )  
  
sleep_dag()
```

Run the above Dag and wait for a while. The TaskInstance will be marked failed after <task_instance_heartbeat_timeout> seconds.

Executor Configuration

Some Executors allow optional per-task configuration - such as the KubernetesExecutor, which lets you set an image to run the task on.

This is achieved via the executor_config argument to a Task or Operator. Here's an example of setting the Docker image for a task that will run on the KubernetesExecutor :

```
MyOperator(...,  
    executor_config={  
        "KubernetesExecutor":  
            {"image": "myCustomDockerImage"}  
    })
```

```
) }
```

The settings you can pass into `executor_config` vary by executor, so read the individual executor documentation in order to see what you can set.